

AN1.12 - Evaluation + Observability (A10)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A10 - Evaluation + Observability **Depends on:** A3 persistence, A4 runtime, A6 integrity, A8 authorization, A9 credits (all closed) **Gate:** G10 - structured, auditable evaluation. **Not** another governance engine.

A10 READS the recorded results of **A6 (the integrity step)**, **A8 (denial steps)** and **A9 (the credit ledger)** from the persisted trace, and **SCORES** them. It never re-checks authorization or integrity. Ownership stays: A6 integrity, A8 authorization, A9 credit accounting, **A10 evaluation + observability**.

1. The pipeline is now complete

Run -> Steps -> ToolCalls -> Evidence -> Output -> Integrity (A6) -> Evaluation (A10) -> Observability (A10)

The runtime, at **every terminal transition** (succeeded, integrity-failed, and every `terminate()` path), calls `evaluationService.evaluate(runId)` **best-effort** - an evaluation failure NEVER changes a run's outcome.

2. What was built - `services/agent-framework/evaluation/`

File	Role
<code>evaluation-service.ts</code>	<code>EvaluationService.evaluate(runId)</code> -> <code>AgentEvaluationResult</code> - the heuristic evaluator (deterministic, no LLM). Once-per-run.
<code>evaluation-store.ts</code> / <code>prisma-evaluation-store.ts</code>	<code>EvaluationStore</code> port + <code>InMemory</code> / <code>Prisma</code> impls.
<code>observability.ts</code>	<code>getRunObservability(runId)</code> - the assembled read model (run + steps + tool calls + evidence + credit history + evaluation + a compact timeline). Pure read; no API route added .
<code>index.ts</code>	<code>barrel</code> + <code>defaultEvaluationStore()</code> (<code>AGENT_CREDIT_INMEMORY=1</code> escape hatch, shared with the ledger).

2.1 The 7 scored dimensions (each 0..1, each with a human-readable basis)

dimension	reads	scoring
completion	<code>run.status</code> , <code>errorCode</code> , denial steps	succeeded -> 1.0 - expected guardrail (<code>permission_denied</code> +denial, <code>insufficient_credits</code>) -> 0.7 - <code>tool_error</code> -> 0.4 - unexpected -> 0.1
tool_selection	executed <code>toolIds</code> vs <code>definition.tools</code> + the type's <code>defaultTools</code> (registry)	fraction bound x fraction in-defaults, halved if an unauthorized tool was attempted
authorization	A8 denial steps in the trace	no denial -> 1.0 - a denial -> 0.4 ("the guardrail held, but the agent overreached")
evidence	<code>buildLineage(output, trace, registry)</code> (A6's helper, read)	complete lineage to a registered capability -> 1.0 - gaps -> 0.3 - none -> 0.3-0.4

dimension	reads	scoring
integrity	the A6 evaluation-kind step's output.passed (read, not re-run)	true -> 1.0 - false -> 0.0 - not reached -> neutral
limits_respected	run.status, ledger sum vs limits.maxCreditCost	hit a resource ceiling -> 0.5 - over maxCreditCost -> 0.2 - within -> 1.0
groundedness	output.resolved / basis / evidenceIds + lineage	resolved + non-empty basis + complete lineage -> 1.0 - resolved but incomplete -> 0.6 - honest "unresolved" -> 0.5 - no conclusion -> 0.1

compositeScore = weighted mean (completion .25, groundedness .2, evidence .15, integrity .15, tool_selection .1, authorization .1, limits_respected .05).

2.2 Failure analysis + measurable signals

- failureCategory in none | expected_guardrail | provider_failure | credit | integrity | resource_limit | unexpected.
- failureAnalysis - a sentence: terminal status, category, errorCode, errorMessage (truncated), the last step's kind/status/summary.
- measurableSignals - the flat observability payload: stepCount, toolCallCount, executedToolCount, evidenceCount, lineageComplete, lineageGaps, deniedGateCount, integrityPassed, integrityViolationCount, creditsCharged, wallMs, planningPath, specialist, terminalStatus, errorCode, agentType, resolved.

2.3 The owner's evaluation questions - answered

question	signal
Did the run complete correctly?	completion dimension + terminalStatus
Were the selected tools appropriate?	tool_selection dimension
Were all tool calls authorized?	authorization dimension + deniedGateCount
Was evidence sufficient and properly linked?	evidence dimension + lineageComplete / lineageGaps
Did output integrity pass?	integrity dimension + integrityPassed (read from A6)
Were limits/credits respected?	limits_respected dimension + creditsCharged
Did the agent produce a grounded conclusion?	groundedness dimension + resolved
Why did the run succeed/fail?	failureCategory + failureAnalysis
What measurable signals to track?	measurableSignals (structured object, not a string)

3. Persistence - AgentEvaluation (migration GENERATED, NOT APPLIED)

prisma/schema.prisma: +1 model:

```
AgentEvaluation
id, runId @unique, agentId, userId,
terminalStatus, scores (Json), compositeScore,
failureCategory, failureAnalysis, measurableSignals (Json),
evaluatorVersion, createdAt
@@index([userId, createdAt]) @@index([agentId]) @@index([terminalStatus])
```

Immutable, one per run (runId @unique). Migration 20260907130000_add_agent_evaluation/migration.sql - GENERATED via prisma migrate diff (offline), hand-reviewed, header marks it NOT APPLIED. Purely additive: 1

table, no DROP, no ALTER TABLE.

4. G10 proof - npm run validate:agent-evaluation -> 9 passed, 0 failed

SUCCEEDED run: structured evaluation, all 7 dimensions present, each with a non-empty basis (not a bare number); completion: 1, integrity: 1, failureCategory: none, compositeScore >= 0.8; measurableSignals has the expected keys, specialist = MARKET_INTELLIGENCE, integrityPassed = true. Live: composite 1.0, 2 tools, 5 evidence, 5 credits	■
idempotent: evaluate() twice -> identical result	■
credit_limit run: failureCategory: "credit", completion: 0.7, failureAnalysis names it, measurableSignals.errorCode = "insufficient_credits", composite < 0.7	■
integrity-failed run: failureCategory: "integrity", integrity score 0, integrityViolationCount >= 1, composite < 0.6	■
permission-denied run: failureCategory: "expected_guardrail", authorization score 0.4, deniedGateCount >= 1	■
getRunObservability: assembles run + steps + tool calls + evidence + credit entries + evaluation + a structured timeline	■
AGENT_EVALUATION_DIMENSIONS == the 7 locked dimensions	■
migration present; NOT APPLIED; additive-only; unique runId	■
structural: no evaluation file imports checkOutputIntegrity (A6) or the authorization-service (A8) - it reads buildLineage only	■

4.1 Regression - no gate lost

validate:agent-contracts 38/0 - validate:agent-tools 20/0 - validate:agent-run-persistence 17/0 - validate:agent-runtime 9/0 - validate:agent-supervisor 11/0 - validate:agent-integrity 21/0 - validate:agent-memory 19/0 - validate:agent-authorization 17/0 - validate:agent-credit 13/0 - validate:agent-evaluation **9/0**.

The runtime now auto-evaluates every run at its terminal transition (best-effort, in-memory store under AGENT_CREDIT_INMEMORY=1 for the harnesses) - the other suites stayed green, confirming evaluation never alters a run's outcome.

5. TypeScript

npx tsc --noEmit: **0 errors** in the framework code + the new AgentEvaluation model. Repo-wide 77, all in the stale generated .next/dev/types/validator.ts (gitignored, pre-existing).

6. Files changed

Added (6):

```
frontend/services/agent-framework/evaluation/evaluation-service.ts
frontend/services/agent-framework/evaluation/evaluation-store.ts
frontend/services/agent-framework/evaluation/prisma-evaluation-store.ts
frontend/services/agent-framework/evaluation/observability.ts
frontend/services/agent-framework/evaluation/index.ts
frontend/prisma/migrations/20260907130000_add_agent_evaluation/migration.sql (GENERATED + REVIEWED, NOT AP
frontend/scripts/validate-agent-evaluation.ts
```

Modified (3):

```
frontend/types/agent-framework/agent-run-contract.ts + AgentEvaluationDimension / RunFailureCategory / Age
frontend/prisma/schema.prisma + 1 model (appended)
frontend/services/agent-framework/runtime/agent-runtime.ts finalize() best-effort evaluate at every termin
frontend/package.json + "validate:agent-evaluation"
```

Not touched: A6 integrity checker, A8 authorization service, A9 ledger logic, contracts A1-A9 behaviour, legacy agents UI.
No API routes.

7. Non-goals honoured

- **Not another authorization or integrity engine** - A10 reads their recorded verdicts; the structural test proves no import of the A6 checker or A8 authorizer.
- Evaluation is **structured** (scores[] + measurableSignals object), never a log string.
- No API route, no dashboard UI (the getRunObservability read model is ready for one).
- No new agents / tools / engines. No LLM in the evaluator.

8. Status

A10 complete. Every terminal run is evaluated into a structured, auditable AgentEvaluation - 7 scored dimensions each with a human-readable basis, a composite score, a failure category + analysis, and a flat measurableSignals payload. getRunObservability(runId) assembles the whole Run -> ... -> Evaluation model for a future route/dashboard. The evaluator reads A6/A8/A9's recorded results; it re-checks nothing. Migration is generated + reviewed, **not applied**.

The foundation (A1-A10) is complete. Next per AN1.1 sec 7: **A11-A13** - the three real agents (Research, Market Intelligence, Strategy Research).

G10 review requested - schema + migration.sql + the evaluator.

End AN1.12.